University of Liberal Arts Bangladesh

Department of Computer Science & Engineering

CSE 2201 : Algorithms

Lecture 12:
Greedy Problems – Bin Packing

Atanu Shuvam Roy

Lecturer,
Dept. of CSE, ULAB, Dhaka

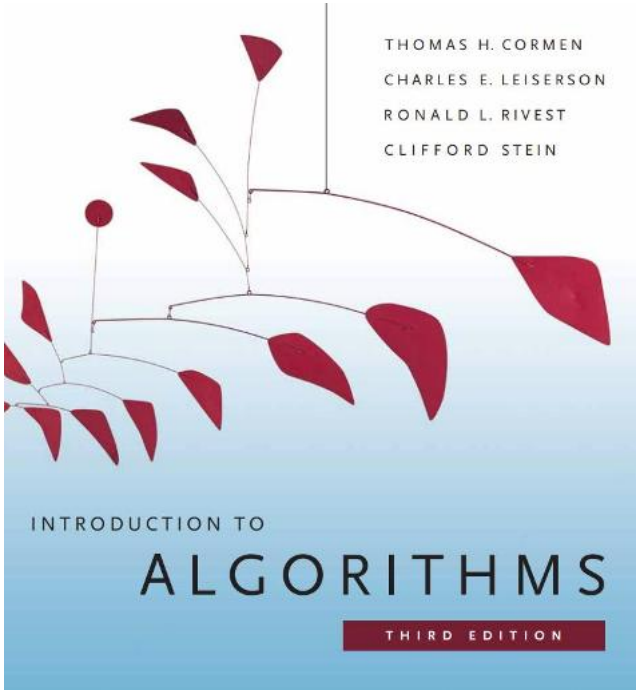
EMAIL:
atanu.shuvam@ulab.edu.bd

Class Code
GVWTHYJV

Fall 2025

Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



Distribution of Marks

Assessments	%
Mid Term Examination	25
Final Examination	40
Attendance & class performance	10
Assignment & Quizzes	25
Total	100

Today's Contents

- **Knapsack Problem**
- **Huffman Coding**

Greedy Algorithms

Part - 3 : Knapsack Problem

Knapsack Problem

1. 0/1 Knapsack

– Either take all of an item or none.

2. Fractional Knapsack

– You can take fractional amount of an item.

Concept of Knapsack Problem

A Set of Items are given where,

- **No of Items = n = [X1, X2, X3, ….. Xn]**
- **Weight of Xi = Wi**
- **Profit of Xi = Pi**
- **Knapsack Size = m**

Goal:

- **Maximize** the Profit **ΣPiXi** within **ΣWiXi <= m**

Fractional Knapsack

10 oz., $1,000

Max Weight: 400 oz.

100 oz., $2,000

300 oz., $4,000

1 oz., $5,000

200 oz., $5,000

Fractional Knapsack problem

ITEM	PRICE	WEIGHT
X1	25	18
X2	24	15
X3	15	10

Total Item , n = 3

Knapsack Size , m = 20

Three Cases:

1. **Maximum Price**
2. **Minimum Weight**
3. **Maximum Pi/Wi**

X1	X2	X3	Σ WiXi	Σ PiXi

Fractional Knapsack problem

ITEM	PRICE	WEIGHT
X1	25	18
X2	24	15
X3	15	10

Total Item , n = 3

Knapsack Size , m = 20

Three Cases:

1. **Maximum Price**
2. **Minimum Weight**
3. **Maximum Pi/Wi**

X1	X2	X3	Σ WiXi	Σ PiXi

Case – 1:

Item 1 : X1 (1)
Profit = 25
m = 20 – 18 = 2

Item 2 : X2 (2/15)
Profit = 24 * (2/15)
= 3.2
m = 2– 2 = 0

Total Profit
= 25 + 3.2
= 28.2

Fractional Knapsack problem

ITEM	PRICE	WEIGHT
X1	25	18
X2	24	15
X3	15	10

Total Item , n = 3

Knapsack Size , m = 20

Three Cases:

1. **Maximum Price**
2. **Minimum Weight**
3. **Maximum Pi/Wi**

X1	X2	X3	Σ WiXi	Σ PiXi
1	2/15	0	20	28.2

Case – 2:

Item 1 : X3 (1)
Profit = 15
m = 20 – 10 = 10

Item 2 : X2 (10/15)
Profit = 24 * (2/3)
= 16
m = 10– 10 = 0

Total Profit
= 15 + 16
= 31

Fractional Knapsack problem

ITEM	PRICE	WEIGHT
X1	25	18
X2	24	15
X3	15	10

Total Item , n = 3

Knapsack Size , m = 20

Three Cases:

1. **Maximum Price**
2. **Minimum Weight**
3. **Maximum Pi/Wi**

X1	X2	X3	Σ WiXi	Σ PiXi
1	2/15	0	20	28.2
0	10/15	1	20	31

Case – 3: (Pi /Wi)

X1 = P1/W1
= 25/18 = 1.39

X2 = P2/W2
= 24/15 = 1.6

X3 = P3 / W3
= 15 / 10 = 1.5

Item 1 : X2 (1)
Profit = 24
m = 20 – 15 = 5

Item 2 : X3 (5/10)
Profit = 15 * (1/2)
= 7.5
m = 10– 10 = 0

Total Profit
= 24 + 7.5
= 31.5

Fractional Knapsack problem

ITEM	PRICE	WEIGHT
X1	25	18
X2	24	15
X3	15	10

Total Item , n = 3

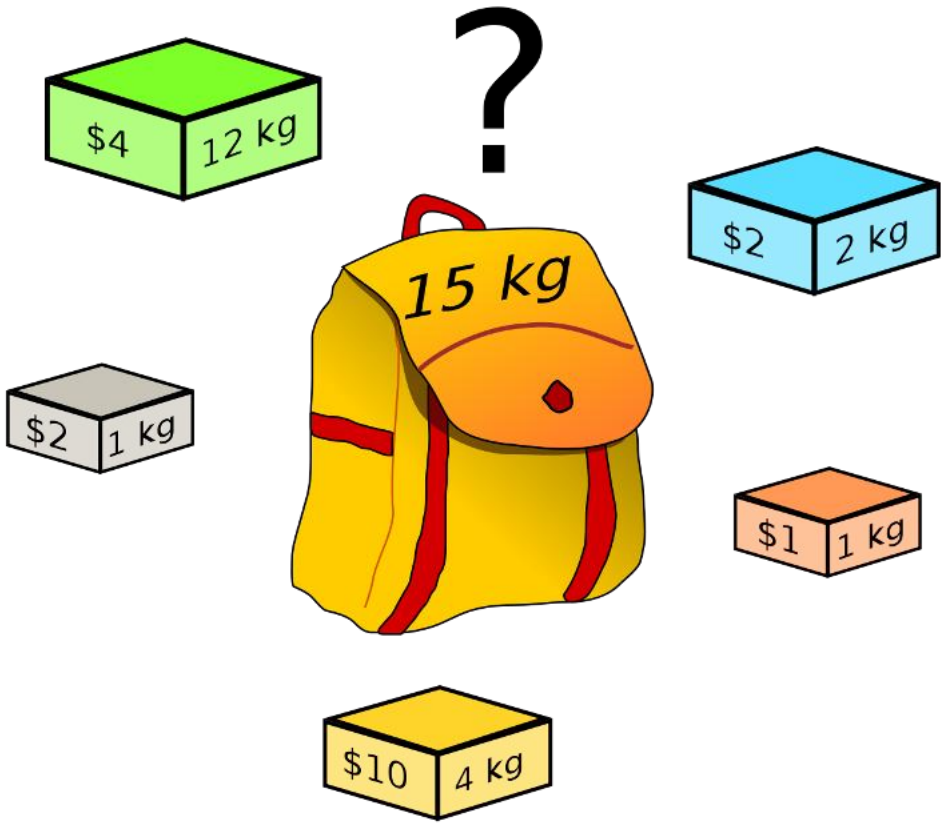
Knapsack Size , m = 20

Three Cases:

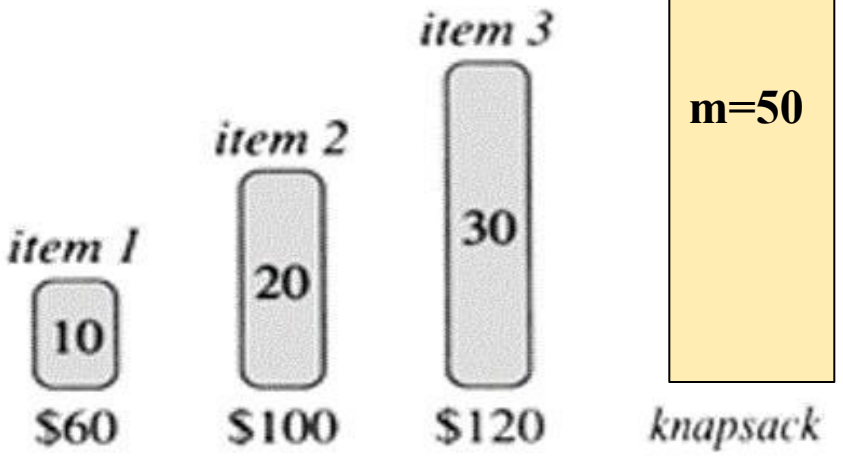
1. **Maximum Price**
2. **Minimum Weight**
3. **Maximum Pi/Wi**

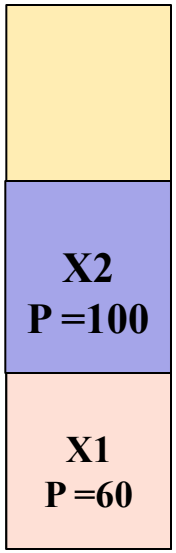
X1	X2	X3	Σ WiXi	Σ PiXi
1	2/15	0	20	28.2
0	10/15	1	20	31
0	1	1/2	20	31.5

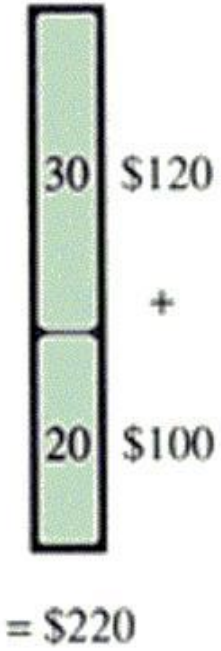
0/1 Knapsack Problem



0/1 Knapsack







x1 = p1/w1
= 60/10
= 6

x2 = p2/w2
= 100/20
= 5

x3 = p3/w3
= 120/30
= 4

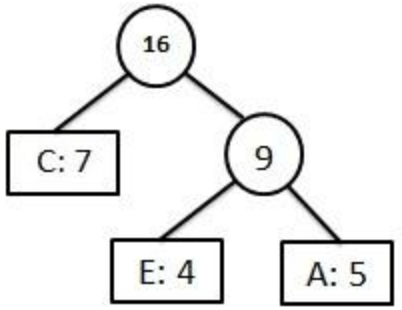
Profit = 60 + 100
= 160

Drawbacks of Greedy Methods

Does not always find the optimal solution for some problems.

Greedy Algorithms

Part - 4 : Huffman Coding



Huffman Codes

- Widely used technique for **data compression**
- Assume the data to be a sequence of characters
- Looking for an **effective way of storing the data**
- ***Binary character code***
- Uniquely represents a character by a **binary string**

Two Ways to Compress data

1. Fixed-Length Codewords

2. Variable-Length Codewords

Example

Suppose you have to compress a data file containing **100,000** characters

The data file only has **6** characters **(a, b, c, d, e, f)**

Characters	a	b	c	d	e	f
Frequency (thousands)	45	13	12	16	9	5

1. Fixed-Length Codes

Data file containing **100,000** characters

Characters	a	b	c	d	e	f
Frequency (thousands)	45	13	12	16	9	5

- 3 bits needed
- a = 000, b = 001, c = 010, d = 011, e = 100, f = 101
- Requires: **100,000 * 3 = 300,000** bits

2. Huffman Codes

- **Idea:**
  - **Variable Length** Codes
  - Use the frequencies of occurrence of characters to build a optimal way of representing each character

Constructing a Huffman Code

- **Idea:**

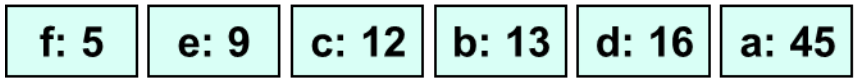
1. Arranging the characters in order of their frequencies

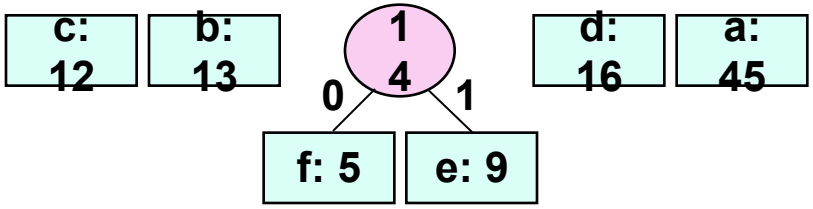
Ascending

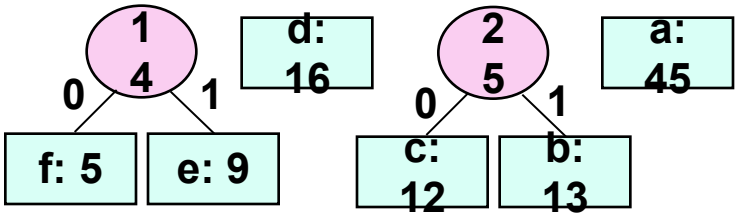
f: 5	e: 9	c: 12	b: 13	d: 16	a: 45

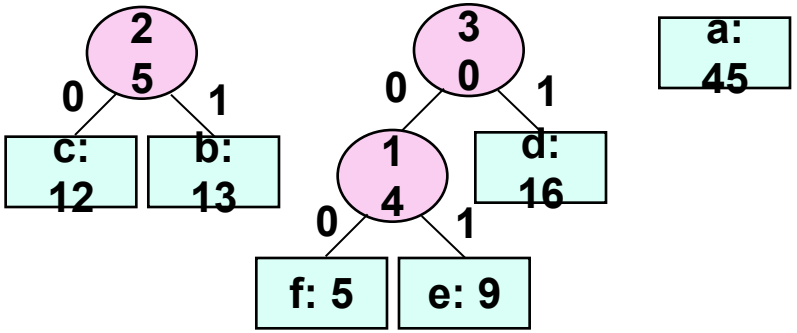
2. Add Two minimum frequencies and place them to the right position

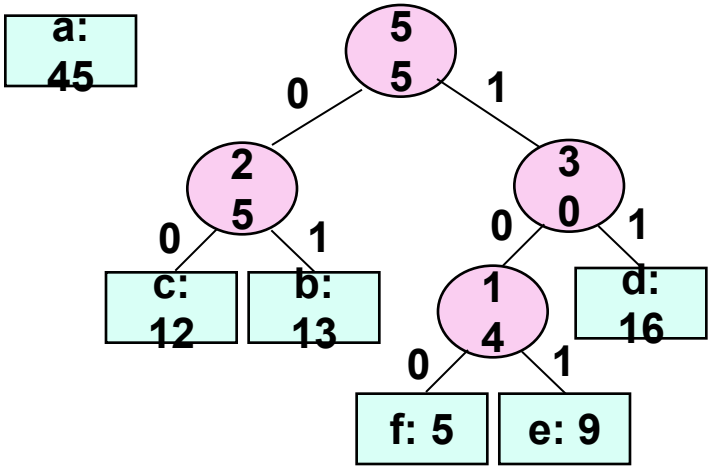
Example

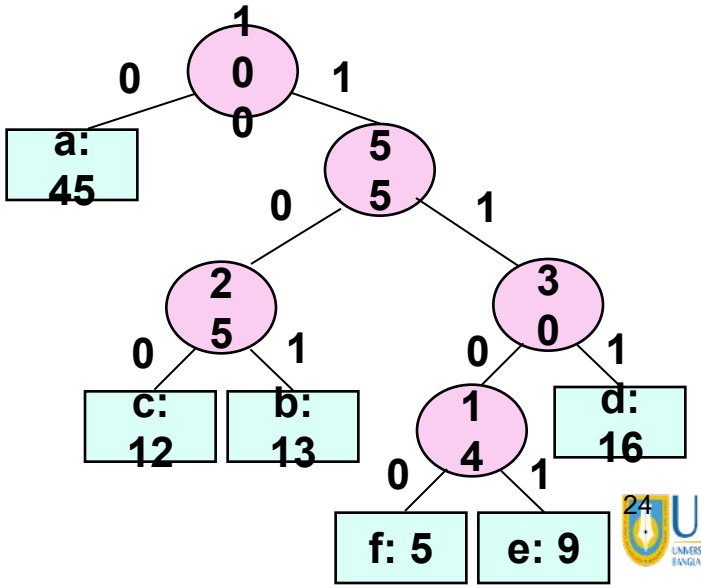

f: 5
e: 9
c: 12
b: 13
d: 16
a: 45


c: 12
b: 13
14
0
1
f: 5
e: 9
d: 16
a: 45

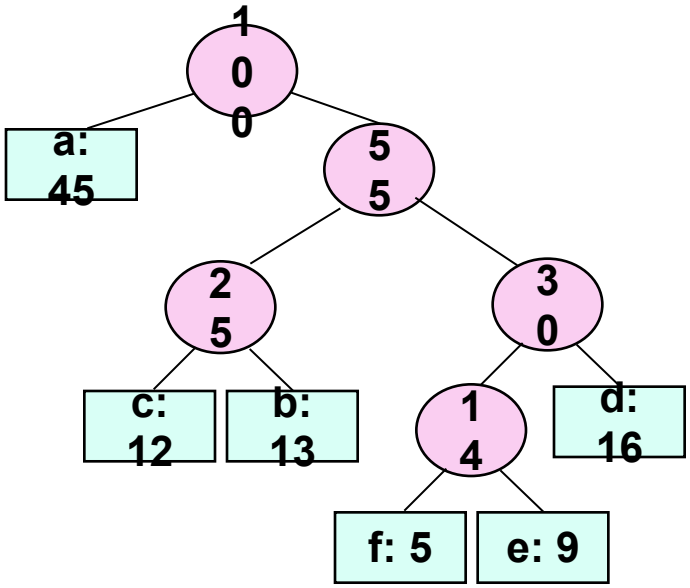

14
0
1
f: 5
e: 9
d: 16
25
0
1
c: 12
b: 13
a: 45


25
0
1
c: 12
b: 13
30
0
1
14
0
1
f: 5
e: 9
d: 16
a: 45


a: 45
55
0
1
25
0
1
c: 12
b: 13
30
0
1
14
0
1
f: 5
e: 9
d: 16

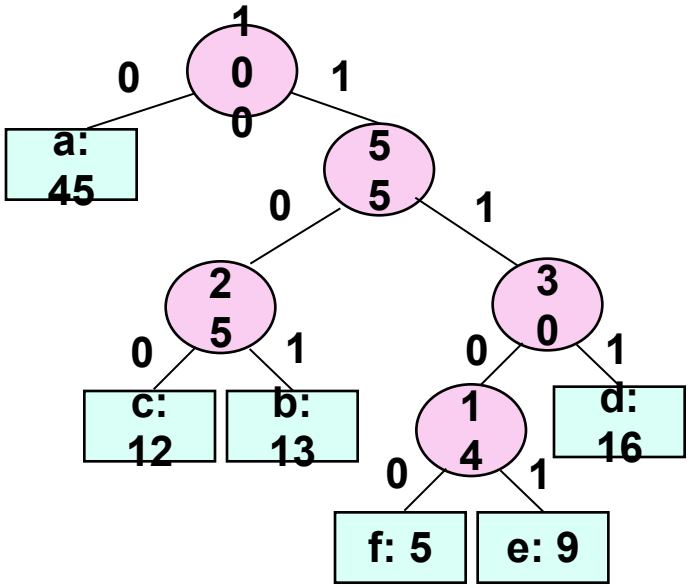

100
0
1
a: 45
55
0
1
25
0
1
c: 12
b: 13
30
0
1
14
0
1
f: 5
e: 9
d: 16

Codewords



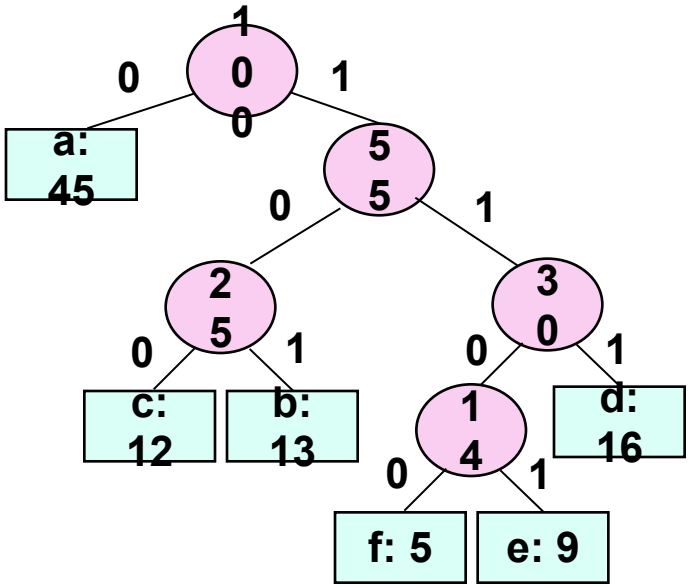
Character	Codeword
a	
b	
c	
d	
e	
f	

Codewords



Character	Codeword
a	
b	
c	
d	
e	
f	

Codewords



Character	Codeword
a	0
b	101
c	100
d	111
e	1101
f	1100

Variable-Length Codes

Data file containing 100,000 characters

Characters	a	b	c	d	e	f
Frequency (thousands)	45	13	12	16	9	5
Variable length Codewords	0	101	100	111	1101	1100

- **((45 * 1) + (13 * 3) + (12 * 3) + (16 * 3) + (9 * 4) + (5 * 4)) X 1,000**

  = 224,000 bits

- Assign **short codewords** to **frequent characters** and **long codewords** to **infrequent characters**

Comparison Ration

Characters	a	b	c	d	e	f
Frequency (thousands)	45	13	12	16	9	5
Fixed Length Codewords	000	001	010	011	100	101
Variable length Codewords	0	101	100	111	1101	1100

$$\left(\frac{300000 - 224000}{300000} \right) \times 100$$

$$= 25.33\%$$

Saves 20% to 90% cost.

Huffman Code: Algorithm

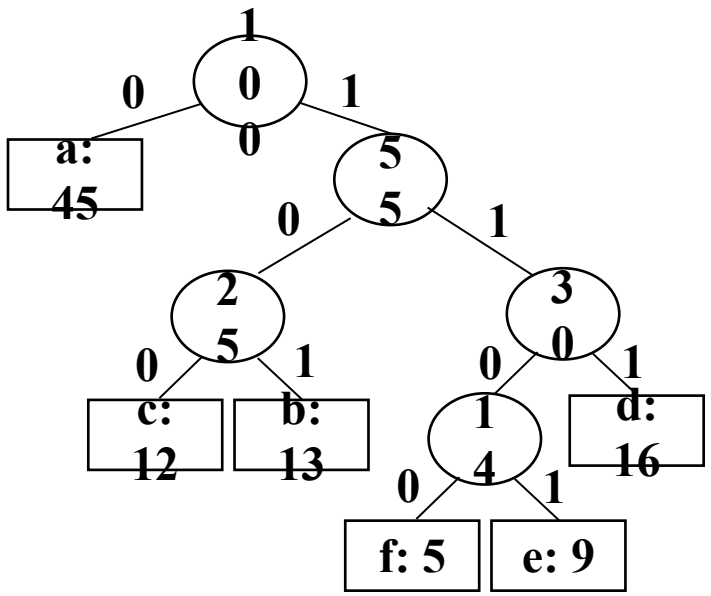
Running time: $O(n \lg n)$

Alg.: HUFFMAN(C)

1. $n \leftarrow |C|$
2. $Q \leftarrow C$ ← $O(n)$
3. **for** $i \leftarrow 1$ **to** $n - 1$
4. **do** allocate a new node z
5. left[z] ← x ← EXTRACT-MIN(Q)
6. right[z] ← y ← EXTRACT-MIN(Q)
7. f[z] ← f[x] + f[y]
8. INSERT (Q, z)
9. **return** EXTRACT-MIN(Q)

lines 3–8: $O(n \lg n)$

Encoding and Decoding



Character	Codeword
a	0
b	101
c	100
d	111
e	1101
f	1100

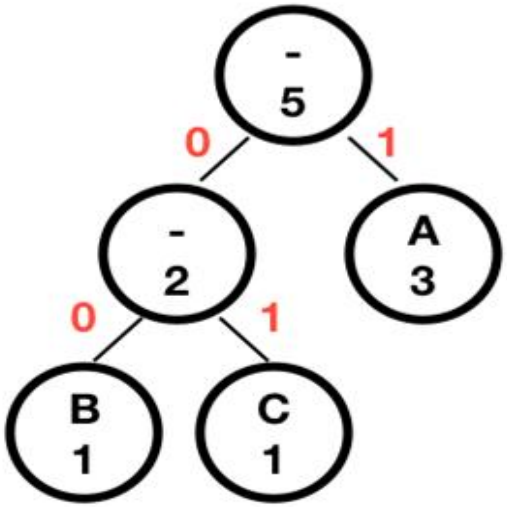
Exercise:

1. Encode : abcde

2. Decode: 10110001111111011100

String to be encoded: ABACA

A = 3 , B = 1, C = 1



Prefix Codes

A - 1
B - 00
C - 01

Reference

- http://coursera.org/learn/algorithms-part1/
- GeeksForGeeks
- YouTube
- LLMs
- Old Lectures of ULAB (Pramanik Sir)

Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh

Department of Computer Science & Engineering